Experiment 9 — Software Testing: Functional and Non-Functional Testing

Software Engineering Laboratory

Name	Pushkar Kumar Mishra
Register Number	24BPS1128
Course	Software Engineering Lab
Date	23rd Sept, 2026
Application under test	Orbital Sentinel — Satellite Telemetry Anomaly Detection System (test build)
Tools used	Selenium IDE (functional testing), Apache JMeter 5.6.3 (load / performance testing)

1. Aim

To design test cases for a web application and execute them using Selenium IDE (functional testing), and to perform a basic load/performance test using Apache JMeter, recording response time and throughput (non-functional testing).

2. Application Under Test

Orbital Sentinel is an early-warning health-monitoring system for satellite fleets (see the SRS and the earlier ER and state-transition experiments). For this experiment, a minimal working build of the system was developed to cover the operator's alert workflow:

- **Login page** — email/password login for the Analyst (Operator) user.
- **Alert dashboard** — a risk-ordered alert list (SRS REQ-16) showing Alert ID, Satellite, Risk Level, Severity, Created At and Status.
- **Acknowledge alert** — moves an alert from *AlertRaised* to *Acknowledged*, as in the state transition diagram (Experiment 5).
- **REST API** — `POST /api/login`, `GET /api/alerts`, `POST /api/alerts/:id/acknowledge`.

The data model uses the entities from the ER diagram (Experiment 4): **Satellite**, **Detection**, **Alert** and **User**. The data is mock data held in memory: 5 satellites, 8 detections and 8 alerts.

Test Environment

Item	Details
Operating system	Windows 11 Home
Application stack	Next.js 15 + React 19 (TypeScript), Node.js 22; production build (`npm run build` + `npm start`)
Application URL	http://localhost:3000
Functional testing tool	Selenium IDE (desktop application)
Performance testing tool	Apache JMeter 5.6.3 on Java 21 (Eclipse Temurin)
Test user	analyst@orbitalsentinel.test / sentinel123 (role: Analyst)

3. Task 1 — Functional Testing using Selenium IDE

3.1 Test Case Table

TC ID	Module	Test Description	Test Steps	Test Data	Expected Result	Actual Result	Status
TC01	Login	Valid login opens the dashboard	1. Open /login 2. Enter email and password 3. Click Login	analyst@orbitalsentinel.test / sentinel123	User is taken to /dashboard; the alert table and the title "Orbital Sentinel - Alert Dashboard" are shown	Login succeeded and the dashboard with the alert table was displayed	**Pass**
TC02	Login	Invalid password is rejected	1. Open /login 2. Enter a valid email and a wrong password 3. Click Login	analyst@orbitalsentinel.test / wrongpass	Error "Invalid email or password" is shown; user stays on /login	Error "Invalid email or password" was displayed; the Login button was still present	**Pass**
TC03	Login	Empty fields are validated	1. Open /login 2. Click Login with both fields empty	(blank)	Error "Email and password are required" is shown	Not executed (see 3.3)	**Not executed**
TC04	Dashboard	Alerts are listed in risk order (REQ-16)	1. Log in 2. Check the alert table	8 mock alerts	"8 alerts" shown; row 1 is ALT-001, risk "10 (CRITICAL)"; last row is ALT-005	Not executed as a separate script (see 3.3)	**Not executed**
TC05	Alert	Acknowledging an alert updates its status	1. Log in 2. Click Acknowledge on ALT-001	ALT-001	Status changes from AlertRaised to Acknowledged; the Acknowledge button becomes disabled	ALT-001 status changed to Acknowledged and its button was disabled; the other alerts stayed AlertRaised	**Pass**
TC06	Session	Logout, then direct dashboard access is blocked	1. Log in 2. Click Logout 3. Open /dashboard directly	—	Login page is shown after logout; /dashboard redirects to /login	Not executed (see 3.3)	**Not executed**

3.2 Test Execution Screenshots

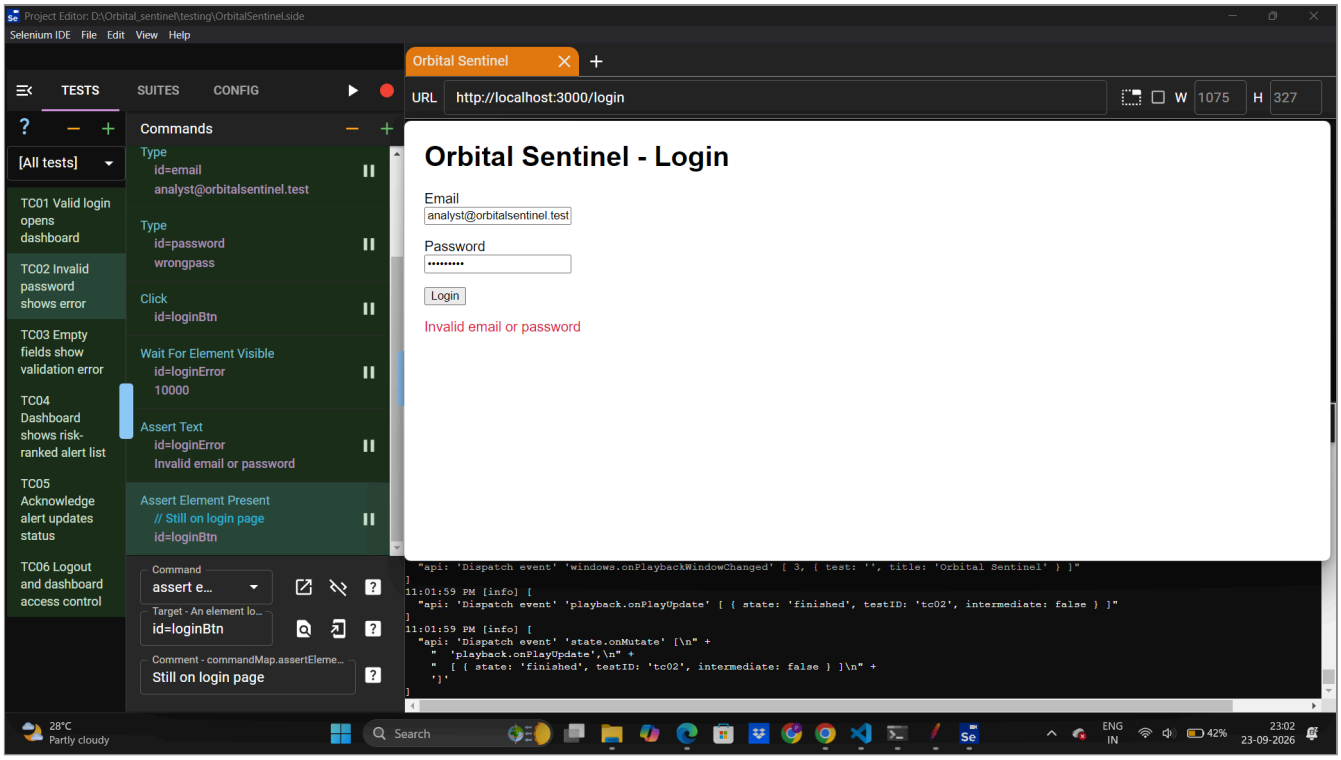


Figure 1: TC02 — invalid password. Selenium IDE shows the executed commands (Type, Click, Wait For Element Visible, Assert Text, Assert Element Present), and the browser shows the "Invalid email or password" message.

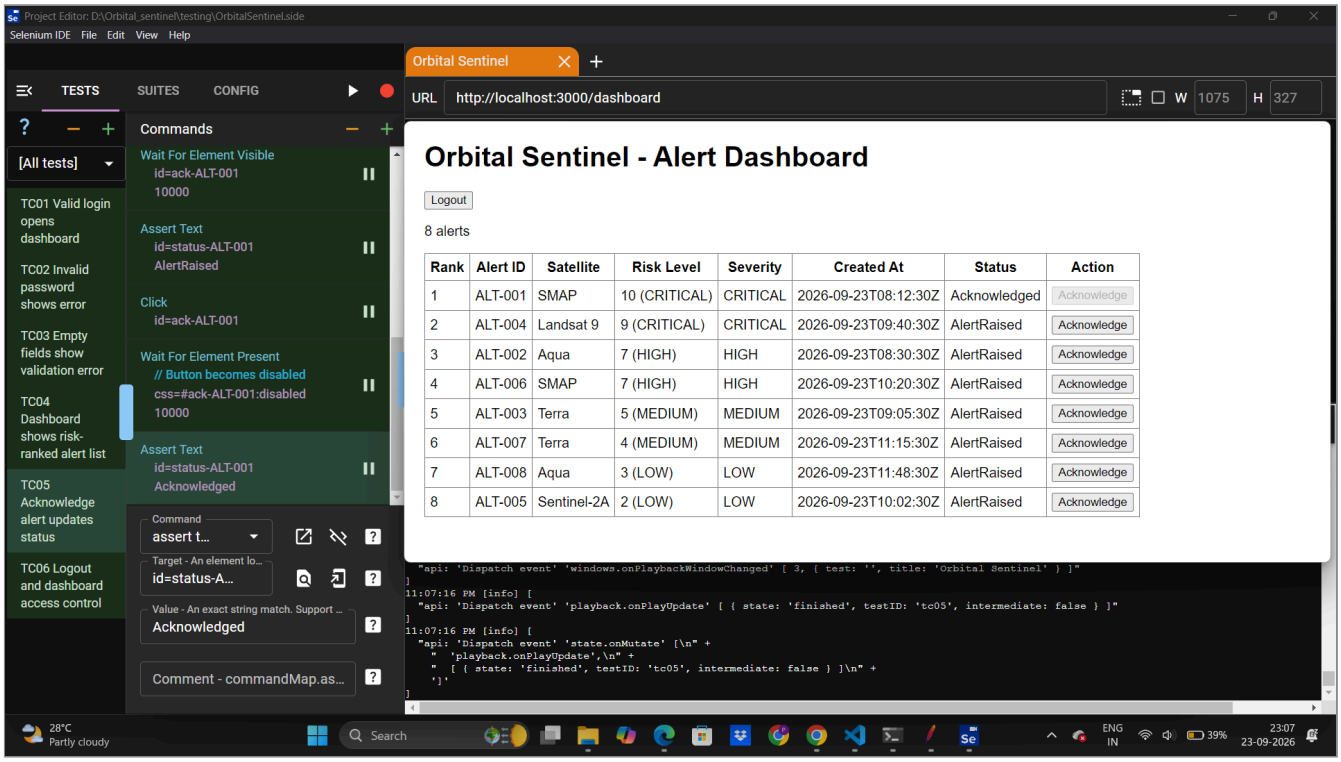


Figure 2: TC05 — acknowledge alert. The assertions confirm ALT-001 changed from AlertRaised to Acknowledged. The dashboard shows ALT-001 acknowledged (button disabled) while the other alerts stay AlertRaised, listed in risk order.

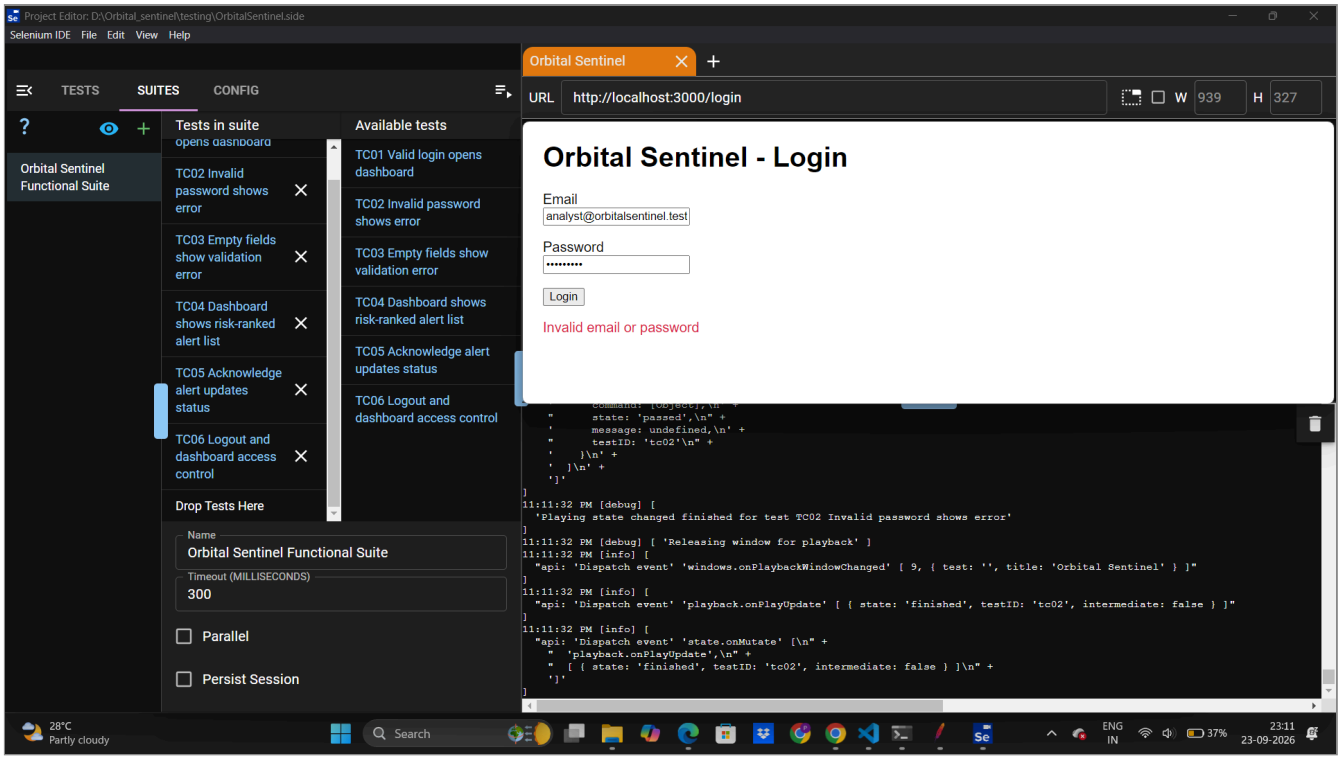


Figure 3: The "Orbital Sentinel Functional Suite" in Selenium IDE, containing all six designed test cases (TC01–TC06). The playback log shows TC02 finishing with state "passed".

3.3 Functional Testing Results

Metric	Count
Test cases designed	6
Test cases executed	3
Passed	3
Failed	0
Not executed	3

All executed test cases (TC01, TC02, TC05) passed; no functional defects were found in the login, error-handling or alert-acknowledgement features. When the whole suite was run, Selenium IDE's desktop playback stopped at the hand-over after TC02 (a tool/playback-window issue, not an application failure), so TC03, TC04 and TC06 were not executed in this session. The dashboard in Figure 2 shows the same risk ordering that TC04 checks (ALT-001 at risk 10 first, ALT-005 at risk 2 last).

4. Task 2 — Non-Functional (Load/Performance) Testing using JMeter

4.1 Test Plan Configuration

Element	Configuration
Thread Group ("Analysts")	50 concurrent users (threads), 10 s ramp-up, 10 loops per user
Requests per iteration	1. POST `/api/login` (JSON body with test credentials) 2. GET `/api/alerts` 3. POST `/api/alerts/ALT-00{1..8}/acknowledge` (random alert)
Total requests	50 users × 10 loops × 3 requests = 1,500 samples
Config elements	HTTP Request Defaults (localhost:3000), HTTP Header Manager (Content-Type: application/json), HTTP Cookie Manager
Assertions	Response code = 200 on every request; response body contains "alertId" for GET /api/alerts
Listeners	Summary Report, Aggregate Report, View Results Tree

4.2 Performance Testing Results

Response times are in milliseconds (ms).

Endpoint	Samples	Average	Median	90% Line	95% Line	99% Line	Min	Max	Std. Dev.	Error %	Throughput
POST /api/login	500	7	6	12	15	23	3	38	4.06	0.00%	50.3/sec
GET /api/alerts	500	6	5	12	15	22	3	24	3.80	0.00%	50.5/sec
POST /api/alerts/:id/acknowledge	500	6	5	12	15	24	3	26	4.00	0.00%	50.5/sec
TOTAL	**1500**	**6**	**5**	**12**	**15**	**23**	**3**	**38**	**3.97**	**0.00%**	**150.7/sec**

Network: 181.03 KB/sec received, 32.57 KB/sec sent (total). Average response size: 440 bytes (login), 2,686.9 bytes (alerts), 564 bytes (acknowledge).

4.3 Performance Test Screenshots

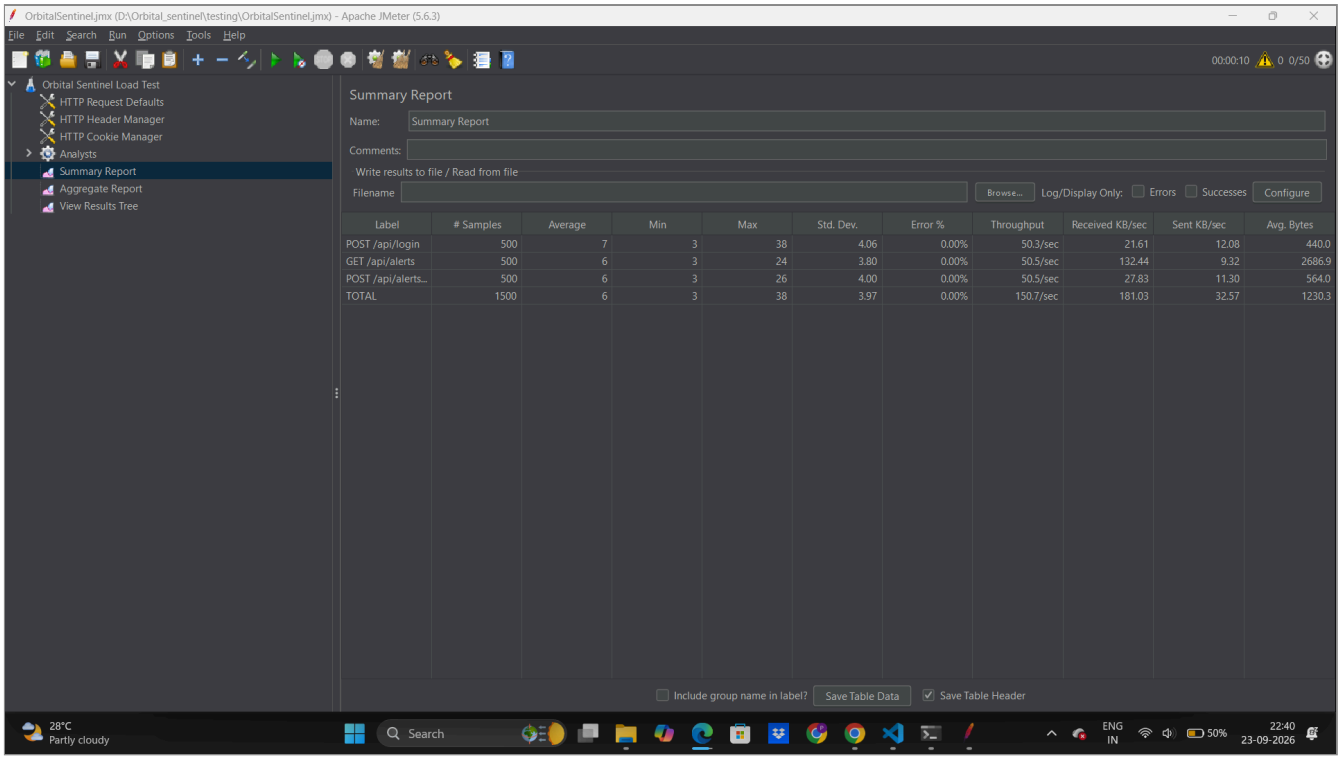


Figure 4: JMeter Summary Report — 1,500 samples, 0.00% errors, average 6 ms, total throughput 150.7 requests/sec.

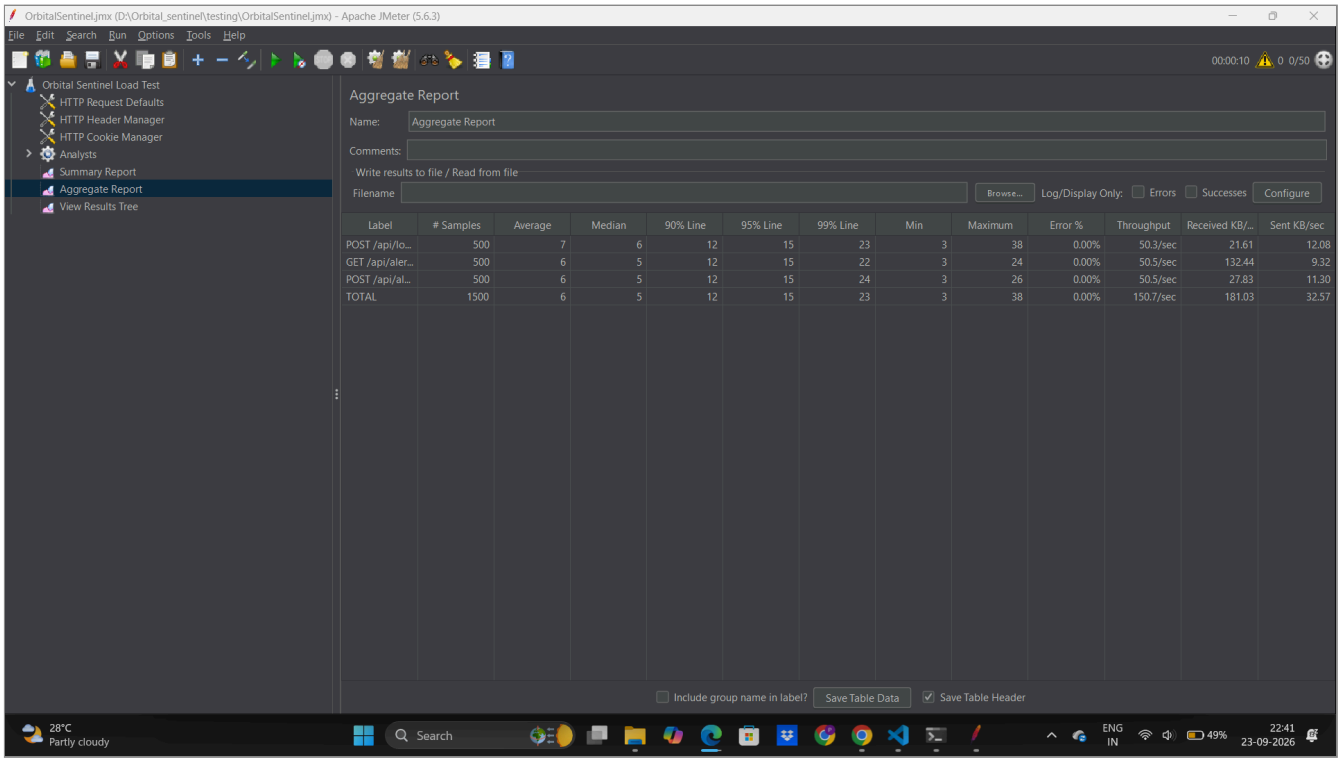


Figure 5: JMeter Aggregate Report — median, 90%, 95% and 99% response-time lines for each endpoint.

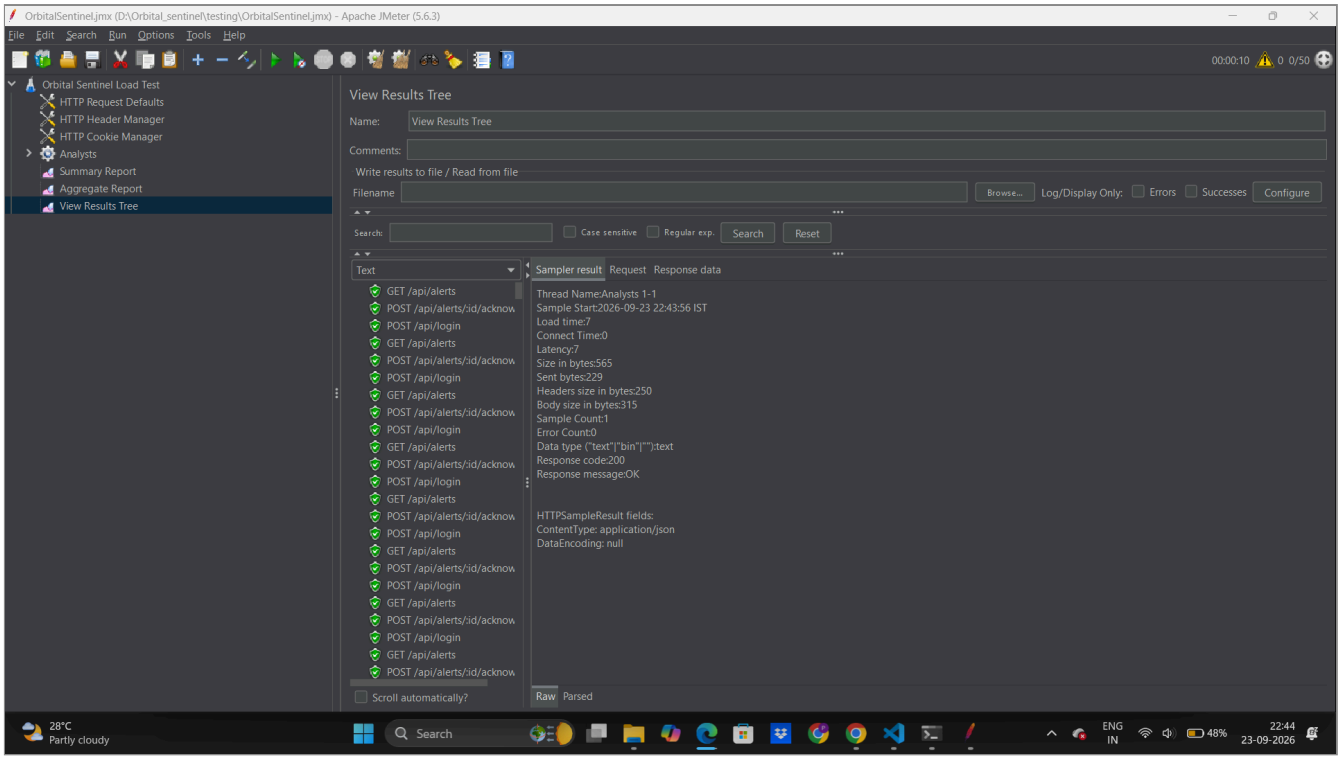


Figure 6: JMeter View Results Tree — every sampled request passed (green), with response code 200 OK. Captured from a repeat run with the same configuration.

4.4 Analysis

- **Response time:** the average response time was 6–7 ms on all endpoints, and 90% of requests completed within 12 ms. The slowest request of the whole test took 38 ms (login).
- **Throughput:** the system handled about 150.7 requests/sec in total (about 50 requests/sec per endpoint) with 50 concurrent users.
- **Reliability:** 0.00% errors across 1,500 requests. Every response returned HTTP 200 and passed its assertions.
- **Stability:** the low standard deviation (about 4 ms) shows consistent response times under load, with no sign of degradation as users ramped up.
- The SRS performance requirement (dashboard updates within about one second) is met with a large margin for these operations. Because the build uses in-memory mock data, these figures are a baseline; they should be measured again once the real database and detection pipeline are integrated.

5. Final Test Report Summary

Area	Result
Functional testing (Selenium IDE)	6 test cases designed; 3 executed, all 3 passed (TC01, TC02, TC05); 0 failed; 3 not executed because of a Selenium IDE suite-playback issue
Defects found	None in the application. Tool issue: Selenium IDE desktop playback stalled between tests when the suite was run.
Load test (JMeter)	50 users, 10 s ramp-up, 10 loops — 1,500 requests
Average / 90th percentile response time	6 ms / 12 ms
Maximum response time	38 ms
Throughput	150.7 requests/sec

Error rate	0.00%
Overall verdict	The executed functional tests passed, and the application met its performance expectations under the tested load.

6. Conclusion

Test cases were designed for the Orbital Sentinel web application and executed with Selenium IDE. Login validation and the alert-acknowledgement workflow (AlertRaised → Acknowledged) behaved as specified in all executed tests. A load test with Apache JMeter on the application's REST API recorded an average response time of 6 ms, a 90th-percentile time of 12 ms, a throughput of 150.7 requests/sec and a 0% error rate with 50 concurrent users. This shows the application is functionally correct for the tested features and responsive under moderate load. Both tools were used successfully to carry out functional and non-functional testing.